

SAPIENZA UNIVERSITÀ DI ROMA

Tecniche di Programmazione Funzionale e Imperativa

*libro del corso:
slide del professor Salvo*

STOP DOING FUNCTIONAL PROGRAMMING

- **FOR LOOPS** WERE NOT SUPPOSED TO BE GIVEN NAMES
- YEARS OF **HOFs** yet NO REAL-WORLD USE FOUND for going higher than **CALLBACKS**
- Wanted to go higher anyway for a laugh? We had a tool for that: It was called **AbstractWidgetLocalizerManagerFactoryBe**
- "Yes please **FOLD** over this collection . Please give me a **CURRIED** function" - Statements dreamed up by the utterly Deranged

LOOK at what **Functional Programmers** have been demanding your Respect for all this time, with all the boilerplate & compilers we built for them **(These are REAL Functions, done by REAL Functional Programmers):**

```
set :: Lens' s a -> a -> s -> s
set in v s = runIdentity (in (const (Identity v)) s)

over :: Lens' s a -> (a -> a) -> s -> s
over in g s = runIdentity (in (Identity . g) s)

view :: Lens' s a -> s -> a
view in v s = getConst (in Const s)
```

```
class Prefunction p where
  clamp :: (a' -> a) -> (b -> b') -> p a b -> p a' b'

  clamp :: (a' -> a) -> p a b -> p a' b
  clamp f = clamp f id

  rmap :: (b -> b') -> p a b -> p a b'
  rmap f = clamp id f
```

```
(>=) :: Maybe a -> (a -> Maybe b) -> Maybe b
(>>) :: Maybe a -> Maybe b -> Maybe b
return :: a -> Maybe a
```

????? ?????? ??????????????????

"Hello I would like a Monad m please"

They have played us for absolute fools

aglaia norza

1 maggio 2026

1	Le basi	3
1.1	Concetti fondamentali	3
1.2	Sintassi, Funzioni e Pattern Matching	4
1.2.1	Definizione e Applicazione	4
1.2.2	Lambda Espressioni e Combinatori	4
1.2.3	Pattern matching	4
1.2.4	Scope locale: <code>let</code> e <code>where</code>	5
1.3	Sistema dei Tipi e Classi	6
1.3.1	Type Signatures	6
1.3.2	Type Classes	6
1.3.3	Definizione di Tipi: <code>type</code> e <code>data</code>	7
1.3.4	Istanze e Derivazione (<code>deriving</code>)	8
1.3.5	Tipabilità e Let-Polymorphism	8
1.3.6	Semantica Denotazionale: ricorsione e punti fissi	9
1.4	Strutture Dati Predefinite	10
1.4.1	Tipi di Base e Tuple	10
1.4.2	Liste	10
1.5	Funzion(al)i di Ordine Superiore	12
1.5.1	Famiglia dei <code>fold</code> (schemi di ricorsione)	12
1.6	Alcuni esempi di stile funzionale	13
2	Lambda Calcolo	15
2.1	Sintassi e Computazione	15
2.2	Variabili e Combinatori	15

2.2.1	Combinatori noti e strutture di controllo	16
2.3	Ricorsione e Combinatori di Punto Fisso	16
3	Temi avanzati di programmazione funzionale	18
3.1	Fusion law per foldr	18
3.2	TODO: tutto il resto	19
3.3	Funtori e applicativi	19
3.3.1	Functor laws	21
3.4	Applicativi	21
3.4.1	Liste come applicativi	22

1.1 Concetti fondamentali

- **Valutazione lazy:** Haskell valuta le espressioni solo quando è strettamente necessario (permette di gestire agevolmente strutture dati infinite)
- **Trasparenza referenziale:** non essendoci assegnazioni o memoria mutabile (assenza di side-effects), una variabile identifica sempre lo stesso oggetto (permette di manipolare algebricamente i programmi)
- **Polimorfismo e type inference:** molte funzioni operano su tipi generici (es. `a`, `b`). Il compilatore inferisce automaticamente il *tipo principale* (il più generale possibile); tutti gli altri tipi corretti sono sue istanze.
- **Non-terminazione, Bottom e undefined:** in Haskell, ogni tipo possiede implicitamente un valore speciale (chiamato *bottom*, indicato matematicamente con $\perp$) che rappresenta un'eccezione fatale o una computazione che entra in un loop infinito
 - **la costante undefined:** è la materializzazione a livello di codice del valore $\perp$ - poiché una computazione che fallisce o non termina può avvenire in qualsiasi contesto (mentre si aspetta un numero, un booleano, ecc.), `undefined` possiede il tipo più generale e polimorfo possibile: `a` (ovvero $\forall \alpha. \alpha$)
 - **Il limite del lambda calcolo (Occurs check):** nel lambda calcolo puro, il loop infinito per eccellenza è il combinatorio $\Omega = (\lambda x.xx)(\lambda x.xx)$. In Haskell, l'auto-applicazione `$\lambda x.xx$` **non è tipabile**. Perché `x` possa prendere se stesso come argomento, il suo tipo α dovrebbe essere uguale a una funzione che prende α come parametro e restituisce β ($\alpha = \alpha \rightarrow \beta$). Questo creerebbe un tipo infinito (durante l'inferenza dei tipi, il compilatore esegue un controllo chiamato *Occurs check* che impedisce alla variabile di tipo α di apparire all'interno della sua stessa definizione, rifiutando di conseguenza l'espressione)
 - **Ricorsione esplicita:** poiché il type system ci vieta di creare loop tramite l'auto-applicazione anonima di `$\lambda x.xx$` , in Haskell la non-terminazione si esprime unicamente attraverso la ricorsione esplicita. Definendo ad esempio `omega' = omega'`, creiamo intenzionalmente una computazione divergente che il compilatore accetta, assegnandole il tipo più generale `a`.
- **Ideologia funzionale:** l'ideologia funzionale favorisce l'assemblaggio di funzioni semplici tramite composizione (`.`). Si pensa in termini di trasformazioni globali di strutture dati, per poi raffinare in un secondo momento i colli di bottiglia prestazionali.

1.2 Sintassi, Funzioni e Pattern Matching

1.2.1 Definizione e Applicazione

- l'applicazione di funzione: **associa a sinistra** e si scrive senza virgole (`f x y`).
- **operatori infissi vs prefissi**: le funzioni binarie si usano in modo infisso tra backtick (`10 `div` 2`), gli operatori infissi in modo prefisso tra parentesi (`(+) 3 4`).

1.2.2 Lambda Espressioni e Combinatori

Le funzioni anonime si scrivono con backslash (`\` simula λ).

```

1  -- identita' (combinatore I)
2  i :: t -> t
3  i x = x           -- definizione standard
4  i' = \x -> x      -- lambda espressione
5
6  -- combinatore K (proiettore del primo argomento) (primo assioma di Hilbert)
7  k :: t1 -> t2 -> t1
8  k' = \x y -> x
9
10 -- combinatore S (sostituzione) (secondo assioma di Hilbert)
11 s :: (t2 -> t1 -> t) -> (t2 -> t1) -> t2 -> t
12 s x y z = x z (y z)

```

1.2.3 Pattern matching

Il pattern matching decompone le strutture dati in base alla loro forma sintattica. L'ordine delle clausole è strettamente sequenziale (viene scelta la prima dall'alto che fa matching).

- **mancanza di unificazione**: non si possono usare variabili identiche nel pattern per verificarne l'uguaglianza (es. `f x x = ...` è invalido)
- **underscore (`_`)**: variabile anonima ("don't care") per valori non rilevanti
- **as-pattern (`@`)**: permette di dare un nome all'intera struttura mentre la si decompone (es. `xs@(_:txs)`)
- **guardie (`|`)**: permettono di definire il comportamento di una funzione valutando sequenzialmente una serie di *predicati booleani* (condizioni logiche); a differenza del pattern matching che controlla la *forma* sintattica dei dati, le guardie valutano *valori e relazioni* (es. `x > 0` , `x == y`)
 - l'ultima guardia è tipicamente `otherwise` (che nel Prelude di Haskell è semplicemente un alias per `True`), che funge da caso di default ("catch-all") per assicurarsi che la funzione restituisca sempre un valore.

```

1  confronta x y
2  | x == y    = "i numeri sono uguali" -- supera la mancanza di unificazione
3  | x > y     = "il primo e' maggiore"
4  | otherwise = "il secondo e' maggiore"

```

- **equazioni multiple (definizione per casi)**: il modo più comune di fare pattern matching è definire la funzione scrivendo più "intestazioni" separate; il compilatore le prova dall'alto verso

il basso: se gli argomenti combaciano con la forma (pattern) della riga, esegue quell'equazione, altrimenti controlla la successiva.

```
1 myAnd True True = True
2 myAnd _ _ = False
```

– è meno comune, ma è possibile fare pattern-matching anche nelle *lambda-espressioni*:

```
1 myFst' = \ (x, y) -> x
```

1.2.4 Scope locale: **let** e **where**

- **let ... in**: costrutto standalone per definire variabili/funzioni locali (utile per strutturare/destrutturare tuple nelle ricorsioni)
- **where**: clausola che qualifica la parte destra di una definizione appena conclusa

```
1 fibAux n = let (f, fPrec) = fibAux (n-1) in (f + fPrec, f)
2 -- equivalente a:
3 fibAux' n = (f + fPrec, f) where (f, fPrec) = fibAux' (n-1)
```

differenza tra **let** e **where**

una clausola **where** è visibile da tutte le guardie di una specifica equazione. Un **let** definito dopo il simbolo **=** di una guardia è visibile solo all'interno di quella singola guardia. Per condividere un calcolo tra più guardie, il **where** è obbligatorio.

```
1 -- condivisione tra guardie
2 analizzaRaggio r
3 | area > 100 = "cerchio grande"
4 | area < 10  = "cerchio piccolo"
5 | otherwise = "cerchio medio"
6 where area = pi * r^2 -- 'area' calcolata una volta e visibile a tutti i pipe
7
8 -- inline: (serve il let perche' non c'e' una dichiarazione di funzione)
9 volumeCilindro r h = (let area = pi * r^2 in area) * h
```

1.3 Sistema dei Tipi e Classi

Haskell è **statically type-safe**: se un programma è tipato correttamente, nessun errore di tipo si verifica durante l'esecuzione.

1.3.1 Type Signatures

In Haskell, grazie al meccanismo di type inference, il compilatore è in grado di dedurre automaticamente il tipo più generale per quasi ogni espressione. Tuttavia, è considerata un'ottima pratica (e in alcuni casi più complessi è strettamente necessario) dichiarare esplicitamente il tipo di una funzione subito sopra la sua definizione.

Questo si fa usando l'operatore `::` (che si legge “ha tipo”). Dichiarare esplicitamente i tipi è utile ai fini della leggibilità del codice e permette di “forzare” il tipo desiderato, aiutando il compilatore a individuare eventuali errori logici in modo molto più preciso.

```
1  -- dichiarazione esplicita del tipo: prende due Int e restituisce un Int
2  somma :: Int -> Int -> Int
3  somma x y = x + y
```

1.3.2 Type Classes

Le classi limitano il polimorfismo imponendo vincoli (o “interfacce”) sui tipi. Più che semplici insiemi, rappresentano classificazioni di tipo algebrico: raggruppano i tipi su cui sono definite certe operazioni. Le classi possono formare gerarchie, dove una sottoclasse “estende” la superclasse ereditandone le operazioni e aggiungendone di nuove.

- **Eq (Equality Types)**: comprende i tipi che ammettono l'uguaglianza (`==`) e la sua negazione (`/=`). Tutti i tipi base (`Bool`, `Int`, `Float`, ecc.) sono istanze di `Eq`, così come le liste e tuple costruite su tipi `Eq`

definizione interna di Eq

```
1  class Eq a where
2    (==), (/=) :: a -> a -> Bool
3    -- le classi possono contenere codice (metodi di default)
4    x /= y = not (x == y)
```

- **Ord (Ordered Types)**: sottoclasse di `Eq` - tipi ordinabili che supportano operazioni come `<=`, `<`, `>=`, `>`, `min` e `max`.
 - per definire un'istanza di `Ord`, è sufficiente fornire il codice per l'operatore `<`, il resto può essere interdefinito.
- **Num**: la classe più generica per i tipi numerici. È sottoclasse di `Eq` e `Show`. Implementa operazioni aritmetiche base (`+`, `-`, `*`), oltre a valore assoluto (`abs`) e segno (`signum`).
i numeri interi costanti sono polimorfi rispetto alla classe `Num`:

```
1  > :t 3
2  3 :: Num a => a  -- 3 puo' essere un Int, Integer, Float, ma non un Char
```

- **Altre classi utili:**

- **Show**: tipi che possono essere stampati (convertiti in `String`) tramite il metodo `show`. Se provate a stampare a video il risultato di una funzione che non appartiene a `Show` (es. una lambda anonima come `\x -> x`), l'interprete darà errore.
- **Read**: permette di trasformare una `String` in un valore tramite il metodo `read`
- **Integral**: sottoclasse di `Num` per i numeri interi (es. `Int`, `Integer`); aggiunge le operazioni di divisione intera `div` e modulo `mod`
- **Fractional**: sottoclasse di `Num` per i numeri frazionari (es. `Float`, `Rational`); aggiunge l'operatore di divisione reale `(/)` e `recip`

1.3.2.1 Vincoli di classe

I vincoli di classe si indicano prima del tipo vero e proprio, separati dalla doppia freccia `=>`. Vincoli multipli si racchiudono tra parentesi e indicano che la funzione opera su un tipo che deve soddisfare tutte le classi elencate.

```

1  -- esempio: mcd richiede che il tipo 'a' supporti sia l'ordine (<, ==)
2  -- sia le operazioni aritmetiche (-)
3  mcd :: (Ord a, Num a) => a -> a -> a
4  mcd x y
5    | x == y    = x
6    | x < y     = mcd (y - x) x
7    | otherwise = mcd (x - y) y

```

1.3.3 Definizione di Tipi: `type` e `data`

- **type** (sinonimi): danno nuovi nomi a tipi esistenti (es. `type Casella = (Int, Int)`)
- **data** (tipi algebrici): costruiscono nuovi tipi tramite costruttori finiti, parametrici o ricorsivi. Possono essere dichiarati istanze di classi.

```

1  data SetteNani = Eolo | Pisolo | Brontolo           -- finiti
2  data Either a b = Left a | Right b                 -- unioni disgiunte
3  data Maybe a = Just a | Nothing                    -- gestione fallimenti
4  data List a = Nil | Cons a (List a)                 -- ricorsivi/induttivi
5  data BTree a = Foglia a | Radice (BTree a) (BTree a)

```

1.3.3.1 Il costruttore di tipo `Maybe`

Il tipo `Maybe` è il tipo di computazioni che possono fallire. In Haskell, è il tipo con cui si rappresentano comunemente le eccezioni in maniera sicura e controllata dal sistema di tipi (corrisponde a `optional`)

Un valore di tipo `Maybe` può assumere due forme:

- `Just v`: contiene un valore `v` (la computazione ha avuto successo e restituisce qualcosa)
- `Nothing`: non contiene nulla (rappresenta una computazione indefinita o fallita)

Si può utilizzare per le *funzioni parziali* (che non sono definite per tutti i possibili valori di input (eg divisione per zero)). Tramite il pattern matching, possiamo “spacchettare” il dato per estrarne il contenuto.


```

1  -- definizione del tipo predefinito
2  data Maybe a = Just a | Nothing
3
4  -- 1. costruzione di un Maybe: divisione sicura
5  -- Se il divisore e' 0 restituisce Nothing, altrimenti restituisce Just il
   risultato
6  safediv :: Integral a => a -> a -> Maybe a
7  safediv n 0 = Nothing
8  safediv n m = Just (n `div` m)
9
10 -- 2. destrutturare un Maybe: funzione "inversa" per estrarre il valore
11 extract :: Maybe a -> a
12 extract (Just n) = n
13 extract Nothing = undefined
14
15 -- extract (safediv 30 0) -> *** Exception: Prelude.undefined

```

1.3.4 Istanze e Derivazione (deriving)

La keyword `deriving` è un meccanismo automatico che permette al compilatore di generare le istanze per le classi predefinite senza dover scrivere codice manuale.

- **Eq**: genera un'uguaglianza basata sulla sintassi
- **Ord**: deriva l'ordine lessicografico in base alla sequenza dei costruttori
- **Show/Read**: la conversione usa i nomi dei costruttori come stringhe
- **Enum**: permette di generare liste per enumerazione (es. `[Lun..Ven]`)

```

1  -- istanze automatiche tramite deriving
2  data Giorni = Lun | Mar | Mer | Gio | Ven | Sab | Dom
3              deriving (Eq, Ord, Show, Enum, Read)
4
5  -- esempio di istanza manuale
6  instance Eq Bool where
7      False == False = True
8      True  == True   = True
9      _    == _      = False

```

1.3.5 Tipabilità e Let-Polymorphism

- Non tutto è tipabile in Haskell. Il classico esempio è l'auto-applicazione pura:
 - `omega = \x -> x x` non è tipabile perché richiederebbe un tipo infinito ($t = t \rightarrow t_1$), portando a una teoria dei tipi non decidibile (e quindi a tipi non inferibili dal compilatore)
- Dal punto di vista del calcolo, `let x = N in M` è equivalente all'applicazione di una lambda: `(\x -> M) N`. Tuttavia, il compilatore Haskell tratta i due casi in modo diverso durante l'inferenza dei tipi.
- **Let-Polymorphism**:

Il costrutto `let` permette di assegnare a una variabile un tipo polimorfo che viene “generalizzato” prima di essere usato nel corpo (`in`).

- **tipabile**: `let x = \y -> y in x x` è tipabile.

Haskell vede che x è l'identità ($a \rightarrow a$) e, nel corpo $x \ x$, può istanziare il primo x come $(a \rightarrow a) \rightarrow (a \rightarrow a)$ e il secondo come $(a \rightarrow a)$.

- **non tipabile:** $(\backslash x \rightarrow x \ x) (\backslash y \rightarrow y)$ **non** è tipabile.

Qui il compilatore deve tipare la funzione $\backslash x \rightarrow x \ x$ prima di sapere che x sarà l'identità. Come visto sopra, $\backslash x \rightarrow x \ x$ è intrinsecamente non tipabile.

Lemma 1: Terminazione

Tutti i lambda-termini tipabili nel sistema dei tipi di Haskell (che non usino ricorsione esplicita tramite equazioni) **terminano**.

1.3.6 Semantica Denotazionale: ricorsione e punti fissi

Haskell è un linguaggio dichiarativo: una definizione non è un comando, ma un'equazione. Quando definiamo una funzione ricorsiva f , stiamo scrivendo un'equazione del tipo $f = \mathcal{E}(f)$, dove $\mathcal{E}$ è un'espressione che contiene f stessa.

- **Bottom ($\perp$)** rappresenta il “non-valore”. Indica un calcolo che non termina (loop infinito) o che fallisce (errore). In Haskell, ogni tipo t contiene implicitamente $\perp$.
- Se la definizione è un'equazione, la “semantica” (ovvero la funzione reale prodotta dal compilatore) deve essere una soluzione dell'equazione. Matematicamente, una soluzione di $f = T(f)$ è un **punto fisso** del funzionale T .

Approfondimento: ricorsione e non-terminazione: `omega'`

Prendiamo la funzione

```
1 omega' x = omega' x
2 -- > :t omega'
3 -- omega' :: t -> t1
```

Questa funzione ha il tipo più generico possibile ($t \rightarrow t1$) perché, non terminando mai, non produce mai un valore reale.

Rimuovendo x , otteniamo $\text{omega}' = I(\text{omega}')$, dove I è l'identità.

Qualsiasi funzione f soddisfa $f = I(f)$. Quale scegliere?

- Haskell sceglie la funzione che “fa il minimo indispensabile” per soddisfare l'equazione. Si usa l'ordinamento $\sqsubseteq$ dove $f \sqsubseteq g$ significa che g è “più definita” di f .
- **Costruzione di Kleene (least-fixpoint):**

il computer “costruisce” la funzione partendo dal nulla ($\perp$):

$$f_0 = \perp \quad (\text{non so nulla})$$

$$f_1 = T(f_0) \quad (\text{applico la definizione una volta})$$

$$f_\infty = \text{limite della catena} = \text{lfp}(T)$$

Nel caso di `omega'`, la catena è $\{\perp, \perp, \dots\}$. Il limite è $\perp$. La semantica di `omega'` è dunque la funzione che restituisce sempre “non-valore”.

Il punto fisso è quindi la “semantica” perché è l’unico modo per dare un valore univoco e calcolabile a una definizione circolare. Il *minimo* punto fisso garantisce che il significato sia quello “prodotto” effettivamente dal calcolo ricorsivo.

`myundefined`

Mentre `omega'` è una *funzione* che non termina, possiamo definire un *valore* che non termina:

```
1 myUndefined = myUndefined
2 -- > :t myUndefined
3 -- myUndefined :: a
```

- Poiché `myUndefined` ha tipo `a`, esso **appartiene a tutti i tipi**. È l’implementazione “home-made” della costante `undefined` di Haskell. Rappresenta il valore $\perp$ per ogni possibile tipo del linguaggio.
- Matematicamente, `myUndefined` è il **punto fisso dell’identità**:
 - (1) la definizione $x = x$ corrisponde all’equazione funzionale $x = I(x)$
 - (2) la soluzione di questa equazione è, per definizione, il punto fisso dell’identità I
 - (3) poiché Haskell usa la semantica del minimo punto fisso, e l’identità applicata al valore nullo ($\perp$) restituisce ancora $\perp$, il risultato è la non-terminazione
 - (4) questo spiega perché `myUndefined` ha tipo `a`: non essendo “sceso” verso nessun valore specifico (come un intero o una stringa), rimane una pura astrazione di errore/loop che può “finger” di essere qualsiasi tipo.

Haskell permette quindi di manipolare la non-terminazione come un valore reale. Ogni tipo in Haskell è in realtà “puntato” (*pointed*), ovvero contiene sempre almeno un valore: $\perp$.

1.4 Strutture Dati Predefinite

1.4.1 Tipi di Base e Tuple

- **Booleani**: `Bool` (`True`, `False`). Operatori: `not`, `&&`, `||`.
- **Interi**: `Int` (dimensione fissa), `Integer` (precisione illimitata).
- **Tuple**: n-uple fisse e disomogenee (es. `(String, Bool)`)
offrono le funzioni *su coppie*: `fst`, `snd` (prendono primo e secondo argomento)

1.4.2 Liste

Sequenze di elementi omogenei. Definite dal costruttore vuoto `[]` e dall’operatore infisso `cons` `(:)`. Le stringhe sono `[Char]`.

Zucchero sintattico: `[1,2,3]` equivale a `1:2:3:[]`.

```
1 -- enumerazioni
2 [m..n]           -- range finito (lista con elementi da m a n)
3 [m, n..p]        -- range con step (es. [1, 3..11] dispari)
```

```
4 [m..]           -- lista infinita (lazy)
```

Haskell supporta le **list comprehension** (definizioni set-like del contenuto di una lista)

```
1 -- list comprehension (spesso tradotte internamente in map/filter)
2 [x^2 | x <- [1..5]]           -- quadrati
3 [x | x <- xs, x `mod` 2 == 0]  -- filtri / guardie
4 [(x,y) | x <- xs, y <- ys]    -- prodotto cartesiano
```

1.4.2.1 Funzioni su Liste

- **Estrazione:** `head`, `tail`, `last`, `init`, `(!!)` per estrarre all'indice n

```
1 head [1, 2, 3]  -- risultato: 1
2 tail [1, 2, 3]  -- risultato: [2, 3]
3 last [1, 2, 3]  -- risultato: 3
4 init [1, 2, 3]  -- risultato: [1, 2]
5 [1, 2, 3] !! 1  -- risultato: 2
```

- **Ispezione:** `null`, `length`

```
1 null []          -- risultato: True
2 length [1, 2, 3] -- risultato: 3
```

- **Taglio:** `take n`, `drop n`, `splitAt n` (tupla con `take` e `drop`).

```
1 take 2 [1, 2, 3, 4]  -- risultato: [1, 2]
2 drop 2 [1, 2, 3, 4]  -- risultato: [3, 4]
3 splitAt 2 [1, 2, 3, 4] -- risultato: ([1, 2], [3, 4])
```

- **Generali:** `++` (concatenazione), `reverse`

```
1 [1, 2] ++ [3, 4]  -- risultato: [1, 2, 3, 4]
2 reverse [1, 2, 3] -- risultato: [3, 2, 1]
```

- **Matematica/logica:** `sum`, `minimum`, `maximum`, `and`, `or`, `any p`, `all p` (predicati su almeno uno o tutti)

```
1 sum [1, 2, 3]      -- risultato: 6
2 minimum [5, 2, 9]  -- risultato: 2
3 maximum [5, 2, 9]  -- risultato: 9
4 and [True, False]  -- risultato: False
5 or [True, False]   -- risultato: True
6 any (> 2) [1, 2, 3] -- risultato: True (almeno uno è > 2)
7 all (> 0) [1, 2, 3] -- risultato: True (tutti sono > 0)
```

- **Liste annidate/multiple:** `concat` (appiattisce liste di liste), `zip` (unisce liste in coppie, fermandosi alla più corta)

```

1 concat [[1, 2], [3, 4]] -- risultato: [1, 2, 3, 4]
2 zip [1, 2] ['a', 'b', 'c'] -- risultato: [(1, 'a'), (2, 'b')]

```

1.5 Funzion(al)i di Ordine Superiore

I funzionali sono funzioni che prendono altre funzioni come argomento o le restituiscono come risultato. Favoriscono la **composizionalità**, permettendo di costruire programmi complessi tramite piccoli blocchi riutilizzabili.

- **curryficazione**: si fonda sull'isomorfismo $A \times B \rightarrow C \cong A \rightarrow (B \rightarrow C)$. Le versioni curryficate permettono di passare un solo argomento alla volta, favorendo l'applicazione parziale.
- **η -regola**: è possibile omettere un parametro in una definizione se esso compare identico alla fine di entrambi i lati dell'equazione.

```

1 -- composizione (.) : Equivale a f(g(x))
2 (.) f g x = f (g x)
3
4 -- curry e uncurry (Isomorfismo A x B -> C <==> A -> (B -> C))
5 curry :: ((a, b) -> c) -> a -> b -> c
6 uncurry :: (a -> b -> c) -> (a, b) -> c
7
8 -- flip: inverte l'ordine dei parametri
9 flip :: (a -> b -> c) -> b -> a -> c
10 myFlip f a b = f b a
11
12 -- flip (:) [2, 3] 1
13 -- [1, 2, 3]
14
15 -- map e filter
16 map :: (a -> b) -> [a] -> [b] -- proprieta': map (f . g) = map f . map g
17 filter :: (a -> Bool) -> [a] -> [a]
18
19 -- zipWith: applica f. binaria agli elementi di due liste
20 zipWith :: (a -> b -> c) -> [a] -> [b] -> [c]

```

1.5.1 Famiglia dei **fold** (schemi di ricorsione)

I **fold** astraggono il concetto di ricorsione strutturale sulle liste, sostituendo i costruttori della struttura dati (l'operatore `(:)` e la lista vuota `[]`) con funzioni e valori specifici. Se l'operatore `#` è associativo e ha `e` come elemento neutro, allora `foldl (#) e = foldr (#) e`.

- **foldr (right fold)**: associa a destra; la riduzione avviene “al ritorno” dalle chiamate ricorsive (costruisce espressioni).
 - sostituisce ricorsivamente il costruttore `(:)` con la funzione `f` data e `[]` con il valore base `z`.
 - espansione: `foldr f z [x1, x2, x3]` diventa `x1 'f' (x2 'f' (x3 'f' z))`
 - può terminare anche su liste infinite se la funzione `f` non è stretta nel suo secondo argomento (ovvero se non ha sempre bisogno di valutare la ricorsione per restituire un risultato).

```

1  -- somma con foldr
2  foldr (+) 0 [1, 2, 3]
3  -- valutazione: 1 + (2 + (3 + 0)) = 6
4
5  -- implementazione di map usando foldr
6  myMap f xs = foldr (\x acc -> f x : acc) [] xs

```

- **foldl** (left fold): associa a sinistra; simula un ciclo iterativo passando un accumulatore “in avanti”
 - valuta l’espressione partendo da sinistra e procedendo verso l’interno - valuta tutto solo alla fine della lista
 - espansione: `foldl f z [x1, x2, x3]` diventa `((z 'f' x1) 'f' x2) 'f' x3`
 - *nota bene*: In `foldl`, la funzione prende come primo argomento l’accumulatore, e come secondo l’elemento corrente della lista.

```

1  -- sottrazione con foldl
2  foldl (-) 0 [1, 2, 3]
3  -- valutazione: ((0 - 1) - 2) - 3 = -6
4
5  -- sottrazione con foldr per confronto
6  foldr (-) 0 [1, 2, 3]
7  -- valutazione: 1 - (2 - (3 - 0)) = 2
8
9  -- reverse usando foldl
10 myReverse xs = foldl (\acc x -> x : acc) [] xs

```

- **foldr1** e **foldl1**: varianti che non richiedono un valore iniziale esplicito, poiché usano il primo o l’ultimo elemento come base; *danno errore su lista vuota*.

```

1  -- utile quando non esiste un elemento neutro sensato (es. min/max assoluto)
2  myMinimum = foldr1 min
3  myMinimum [5, 3, 8] -- risultato: 3
4  -- myMinimum []      -- genera errore a runtime

```

1.6 Alcuni esempi di stile funzionale

Lo stile funzionale evita accumulatori ed indici, preferendo la manipolazione globale delle strutture dati.

- **prodotto scalare**: può essere implementato in diversi modi (flessibilità dei funzionali):

```

1  -- accoppia gli elementi, trasforma "*" in una funzione che accetta una tupla
2  -- applica la moltiplicazione agli elementi, e somma i prodotti
3  ps1 xs ys = sum (map (uncurry (*)) (zip xs ys))
4
5  -- crea una lista di funzioni parzialmente applicate
6  -- (da [5, 4] si ottiene [(5*), (4*)])
7  -- zApp/applyL prende la lista di funzioni ^ e la applica sulla seconda lista
8  ps2 xs ys = sum (applyL (map (*) xs) ys)
9
10 -- zipWith fa zip e map in un passo solo

```

```
11 ps3 xs ys = sum (zipWith (*) xs ys)
```

- controllare se una lista è ordinata:

```
1 ordinata xs = and (zipWith (<=) xs (tail xs))
```

- **ricerca su lista** (`findVPH`): si può implementare accoppiando la lista agli indici (`zip [0..] xs`) e filtrando per il valore cercato.

Ideato da Alonzo Church nel 1931, il λ -calcolo nasce per esplicitare il *processo meccanico di calcolo* (sintassi) di una funzione, in contrasto con la visione matematica classica che vede la funzione solo come una relazione statica tra insiemi (semantica).

2.1 Sintassi e Computazione

La sintassi del λ -calcolo è minimalista ed è composta da tre regole:

- **variabile:** $x, y, z \dots$
- **astrazione:** $\lambda x.M$ (corrisponde alla definizione di una funzione con parametro x e corpo M)
- **applicazione:** (MN) (corrisponde all'applicazione della funzione M all'argomento N)

in BNF, quindi, la grammatica del lambda-calcolo è quindi:

$$T ::= V \mid \lambda V.T \mid (T T)$$

Regole di associazione

- **l'applicazione associa a sinistra:** $FN_1N_2 \dots N_n$ equivale a $(\dots (FN_1) \dots N_n)$
- **l'astrazione associa a destra:** $\lambda x_1 \dots x_n.M$ equivale a $\lambda x_1.(\lambda x_2.(\dots (\lambda x_n.M) \dots))$

La computazione avviene tramite la β -riduzione (beta-regola): un termine della forma $(\lambda x.M)N$ si dice *redex* e si riduce sostituendo tutte le occorrenze di x in M con il termine N . Formalmente:

$$(\lambda x.M)N \rightarrow_{\beta} M[N/x]$$

Un termine che non contiene più β -redex è detto in **forma normale** e rappresenta un valore finale (il risultato della computazione).

2.2 Variabili e Combinatori

- **Variabili libere (FV) e legate:** in $\lambda x.M$, la variabile x è *legata* (bound) all'astrazione. Qualsiasi variabile non legata è *libera* (free).
- **α -conversione (alfa-regola):** per evitare la cattura accidentale di variabili libere durante la sostituzione, si possono rinominare le variabili legate (es. $\lambda x.x \equiv \lambda y.y$).

- è buona pratica seguire la “convenzione di Barendregt”, che stabilisce che, all’interno di un termine lambda o di un insieme di termini, tutte le variabili legate devono avere nomi distinti tra loro e diversi da tutte le variabili libere presenti nel termine stesso
- I termini tali che $(FV(M) = \emptyset)$ (λ -termini chiusi), ovvero senza variabili libere, sono chiamati **combinatori**

2.2.1 Combinatori noti e strutture di controllo

Il λ -calcolo puro non ha tipi di dato predefiniti: tutto (booleani, numeri, if-then-else) deve essere codificato tramite funzioni.

- **Identità (I):** $I \equiv \lambda x.x$
- **Cancellatori (booleani):**
 - True (K):** $K \equiv \lambda xy.x$ (prende due argomenti e restituisce il primo)
 - False (O):** $O \equiv \lambda xy.y$ (prende due argomenti e restituisce il secondo)
- **If-Then-Else:** grazie alla definizione dei booleani, la struttura condizionale è codificata come un’applicazione: $\text{if } x \text{ then } M \text{ else } N \equiv xMN$. Se x è True (K) selezionerà M , se è False (O) selezionerà N .
- **Compositori (S):** $S \equiv \lambda xyz.xz(yz)$ (si può dimostrare che $SKK \approx I$).
- **Duplicatori e divergenza:**
 - $\omega \equiv \lambda x.xx$ (auto-applicazione)
 - $\Omega \equiv \omega\omega \rightarrow \omega\omega \rightarrow \dots$ rappresenta la **funzione indefinita**, il loop infinito (non tipabile)

2.2.1.1 Numeri di Church e iteratori

I **Numerali di Church** rappresentano i numeri naturali come funzioni di ordine superiore che *iterano* un’operazione. Il numero n è una funzione che prende una funzione (es. successore) s e un valore di partenza (es. zero) z , e applica s ad z esattamente n volte

$$\underline{n} \equiv \lambda sz.s(s(\dots(sz)\dots)) \quad [n \text{ applicazioni}]$$

- $\underline{0} \equiv \lambda sz.z$ (equivale a `False`)
- $\underline{3} \equiv \lambda sz.s(s(s(z)))$

Per calcolare il successore (+1) di un numero di Church, basta creare una funzione che applica s una volta in più: $\text{succ} \equiv \lambda xsz.s(xsz)$ (o anche $\lambda xsz.xs(s(z))$)

I numerali di Church non sono quindi solo dati, ma **iteratori intrinseci**.

2.3 Ricorsione e Combinatori di Punto Fisso

Per definire funzioni ricorsive complesse il λ -calcolo necessita di un principio di ricorsione generale.

Thm. 1: Combinatore di Punto Fisso

Nel λ -calcolo esiste un termine Y (detto **Combinatore di Punto Fisso**) tale che, per ogni termine M , si ha $YM \rightarrow M(YM)$.

Il combinatore permette di trovare quindi il punto fisso di una funzione (quel valore x tale che $f(x) = x$)

Due combinatori di punto fisso sono:

- **Combinatore di Turing (Y_T):** definito come $Y_T = \theta\theta$ dove $\theta \equiv \lambda xy.y(xxy)$.

$$\begin{aligned} Y_T M &\equiv (\theta\theta)M \\ &\equiv ((\lambda xy.y(xxy))\theta)M \\ &\rightarrow_\beta (\lambda y.y(\theta\theta y))M \\ &\rightarrow_\beta M(\theta\theta M) \\ &\equiv M(Y_T M) \end{aligned}$$

- **Combinatore di Curry (Y_C):** definito come $Y_C \equiv \lambda f.(\lambda x.f(xx))(\lambda x.f(xx))$. Se applicato all'identità I , questo combinatore riduce a Ω (divergenza).

$$\begin{aligned} Y_C M &= (\lambda f.(\lambda x.f(xx))(\lambda x.f(xx)))M \\ &\rightarrow_\beta (\lambda x.M(xx))(\lambda x.M(xx)) \\ &\rightarrow_\beta M((\lambda x.M(xx))(\lambda x.M(xx))) \\ &\equiv M(Y_C M) \end{aligned}$$

(l'ultima equivalenza $\equiv$ vale perché $(\lambda x.M(xx))(\lambda x.M(xx))$ è la forma ridotta di $Y_C M$)

Thm. 2: Tesi di Church-Turing

Ogni funzione calcolabile è λ -definibile.

Temi avanzati di programmazione funzionale

3.1 Fusion law per foldr

La **Fusion Law** per `foldr` è un principio di induzione generalizzato che permette di trasformare la composizione di una funzione con una `foldr` in un'unica `fold`. Essa stabilisce le condizioni per cui vale l'uguaglianza:

$$f . \text{foldr } g \ a = \text{foldr } h \ b$$

L'obiettivo è quindi trovare funzioni o valori `h` e `b` tali che l'applicazione di `f` al risultato di una `foldr` sia equivalente ad una nuova `foldr` che lavora direttamente su di essi - l'utilità computazionale risiede nella capacità di "fondere" due passaggi (una trasformazione `f` applicata dopo un `fold` con `g`) in un unico ciclo sulla struttura dati, evitando allocazioni intermedie.

Thm. 3: Fusion law

Siano `f`, `g`, `h` funzioni e `a`, `b` valori. L'uguaglianza $f . \text{foldr } g \ a = \text{foldr } h \ b$ è garantita se sono soddisfatti i seguenti tre requisiti:

- (1) `f` è stretta ($f \text{ undefined} = \text{undefined}$) (così che valga anche nel caso di liste parziali/indefinite)
- (2) **caso base:** $f \ a = b$ - `f` applicata all'accumulatore iniziale della prima `foldr` (`a`) deve dare come risultato l'accumulatore iniziale della seconda (`b`)
- (3) **passo induttivo** (condizione di fusione): per ogni `x`, `y`, deve valere:

$$f \ (g \ x \ y) = h \ x \ (f \ y)$$
 - in questo modo, l'operazione `h` del nuovo `foldr` riproduce esattamente l'effetto di `g` seguito da `f`

parallelo con un omorfismo

La fusion law può essere interpretata come la dimostrazione che `f` agisce come un **omomorfismo** tra due strutture algebriche di calcolo:

- **preservazione dell'identità:** $f \ a = b$ mappa l'elemento neutro (accumulatore iniziale) del primo dominio in quello del secondo, analogamente a $f(1_A) = 1_B$.
- **preservazione dell'operazione:** La condizione $f(g \ x \ y) = h \ x \ (f \ y)$ ricalca la proprietà omomorfica $f(a \bullet b) = f(a) \circ f(b)$.

- *sorgente* ($g\ x\ y$): rappresenta l'operazione di combinazione di un elemento x con il risultato parziale y tramite la funzione g
- *destinazione* ($h\ x\ (f\ y)$): rappresenta la nuova operazione h che combina x con il risultato già trasformato da f
- **NB:** in $h\ x\ (f\ y)$, l'elemento x non viene trasformato da f - nella struttura della lista, x è un dato "guida" di tipo A , mentre y è il risultato della ricorsione (accumulatore) di tipo B . La fusion law mira a trasformare il *risultato complessivo* del fold, non i singoli elementi contenuti nella lista.

Derivazione formale

Per dimostrare la validità dell'uguaglianza $f . foldr\ g\ a = foldr\ h\ b$, analizziamo separatamente i due lati della relazione applicando i principi di induzione sulla struttura della lista:

Ricordando la definizione di `foldr` basata sulla decomposizione della lista:

- `foldr g a [] = a`
- `foldr g a (x:xs) = g x (foldr g a xs)`

Caso [] (base):

- **sinistra:** $(f . foldr\ g\ a)\ []$
 $= f\ (foldr\ g\ a\ [])\ (def\ di\ .)$
 $= f\ a\ (def\ di\ foldr)$
- **destra:** `foldr h b []`
 $= b\ (def\ di\ foldr)$
- da cui deriva la prima condizione: $f\ a = b$

Caso (x:xs) (induttivo):

- **sinistra:** $(f . foldr\ g\ a)\ (x:xs)$
 $= f\ (foldr\ g\ a\ (x:xs))\ (def\ di\ .)$
 $= f\ (g\ x\ (foldr\ g\ a\ xs))\ (def\ di\ foldr)$
- **destra:** `foldr h b (x:xs)`
 $= h\ x\ (foldr\ h\ b\ xs)\ (def\ di\ foldr)$
 $= h\ x\ (f\ (foldr\ g\ a\ xs))\ (induzione)$
- ponendo $y = foldr\ g\ a\ xs$, deriviamo la condizione di fusione:
 $f\ (g\ x\ y) = h\ x\ (f\ y)$

3.2 TODO: tutto il resto

3.3 Funtori e applicativi

L'idea alla base di un **funtore** è la generalizzazione del funzionale `map`, tipicamente utilizzato per le liste, a tutti i possibili costruttori di tipo.

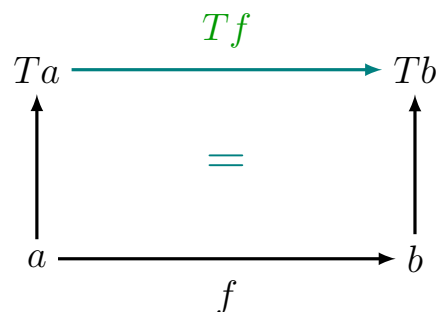
Se si ha:

- una funzione $f :: a \rightarrow b$

- un costruttore di tipo T che trasforma un tipo a in un tipo Ta (come per esempio `[a]` oppure `Maybe a`)

Si può ottenere in modo canonico una funzione

$$Tf :: Ta \rightarrow Tb$$



Nel mondo categoriale, il concetto di “funtore” descrive un passaggio tra due categorie: dati due oggetti (tipi come a o b), un funtore agisce come un costruttore di tipo (T) - prende un oggetto a nella categoria C e lo trasforma in un oggetto Ta nella categoria D .

- il punto importante è che un funtore non mappa solo i tipi, ma anche le relazioni tra essi: se esiste una funzione $f : a \rightarrow b$, il funtore deve fornire un modo per ottenere una funzione corrispondente Tf che operi sui tipi $Ta \rightarrow Tb$
- un funtore è quindi una **mappa che preserva la struttura**

La classe `Functor` richiede che sia definita una funzione `fmap`, che rende commutativo il passaggio dai tipi base ai tipi “inscatolati”

```

1 class Functor t where
2   fmap :: (a -> b) -> t a -> t b

```

- il tipo di `fmap` è parametrico, oltre che nei tipi `a` e `b`, anche rispetto a un **costruttore di tipo** `t` (si vede che `t` deve essere un costruttore di tipo perché si applica ad altri tipi)

Esempi

Alcuni esempi noti di istanze di funtori sono:

- le liste

```

1 class Functor [] where
2   fmap = map

```

- il tipo `Maybe`

```

1 class Functor Maybe where
2   fmap f Nothing = Nothing
3   fmap f (Just x) = Just (f x)

```

■ gli alberi binari

```
1 class Functor BinTree where
2   fmap f (F x) = F (f x)
3   fmap f (R r lft rgt) =
4     R (f r) (fmap f lft) (fmap f rgt)
```

Definire un tipo come istanza di `Functor` permette di scrivere codice generico che funziona su diverse strutture dato.

Per esempio, si può definire una funzione `inc` che incrementa i valori all'interno di qualsiasi funtore:

```
1 inc :: Functor t => t Int -> t Int
2 inc = fmap (+1)
```

3.3.1 Functor laws

Affinché un funtore sia definito correttamente, è necessario che rispetti le due functor laws:

- (1) `fmap id = id`
- (2) `fmap (f . g) = fmap f . fmap g`

Nota bene: queste leggi non possono essere verificate dal type-checker, ma sono responsabilità del programmatore Haskell !

Funtori come omomorfismi

Possiamo notare, osservando le *functor laws*, che un funtore è effettivamente un **omomorfismo tra categorie** (un omomorfismo “classico” preserva l'operazione di gruppo, mentre un funtore preserva la “struttura” di una categoria, ovvero gli oggetti - i tipi -, e i morfismi - le funzioni.)

3.4 Applicativi

Un normale funtore permette di usare `fmap` con un solo argomento, ma è naturale voler generalizzare: si vuole considerare una forma astratta di applicazione, che generalizzi l'usuale applicazione di funzioni. Per esempio, si vuole poter propagare un fallimento nel caso di `Maybe`, o applicare una funzione n -aria a valori “inscatolati” senza dover istanziare n funtori.

Haskell definisce quindi la classe `Applicative`.

```
1 class Functor t => Applicative t where
2   pure :: a -> t a
3   (<*>) :: t (a -> b) -> t a -> t b
```

Un applicativo estende un funtore, e richiede due operatori fondamentali:

- `pure` : prende un valore “normale” e lo immerge nel contesto (lo “impacchetta”)
 - per esempio, per `Maybe`, `pure x` è `Just x`

- `<*>` (*splat* o *apply*): prende una funzione “impacchettata” in un contesto `(t (a -> b))` e un valore anch’esso in un contesto `(t a)`, “scarta” concettualmente entrambi i contesti, applica la funzione al valore (gestendo gli effetti del contesto), produce il risultato e lo “incarta” nel contesto `(t b)`

Esempio: `Maybe`

Possiamo istanziare `Maybe` anche come applicativo:

```
1 instance Applicative Maybe where
2   pure = Just
3   Nothing <*> _ = Nothing -- se uno dei due è Nothing, entrambi sono Nothing
4   (Just g) <*> mx = fmap g mx
5   -- <*> prende una funzione e un valore in un contesto, quindi
6   -- per es. Just (+3) <*> Just 2 = Just 5
```

3.4.1 Liste come applicativi

L’istanza del Prelude delle liste non si comporta come una `zipWith` nell’applicare le funzioni presenti in una lista, ma modella un non-determinismo. Una lista non viene vista come una sequenza, ma come un valore che può assumere più risultati possibili contemporaneamente. Quindi, quando si applica una lista di funzioni ad una lista di valori, l’applicativo contiene tutte le combinazioni possibili (prodotto cartesiano).

```
1 instance Applicative [] where
2   -- pure : a -> [a]
3   pure x = [x]
4
5   -- <*> : [a -> b] -> [a] -> [b]
6   gs <*> xs = [g x | g <- gs, x <- xs]
7   -- ^ prodotto cartesiano
```

Se eseguiamo quindi `pure (*) <*> [1,2] <*> [3,4]`:

- `pure (*)` crea `[(*)]`
- `[(*)] <*> [1,2]` produce una lista di funzioni `[(1*), (2*)]`
- `[(1*), (2*)] <*> [3,4]` applica entrambe le funzioni ad entrambi i numeri
si ottiene quindi `1*3, 1*4, 2*3, 2*4 → [3, 4, 5, 6]`

A volte si vuole però che le liste si comportino come `zipWith` - per questo, haskell introduce `ZipList`